Clase 3 de 5

Google Cloud | The LAB

Transformación y capa de métricas

Ingeniería y Análisis de Datos en Google Cloud

UTN · Facultad Regional Rosario

20 de octubre de 2026

Lo que vimos en la clase 2

El schema se declara

Lo confiable se tipa al cargar. Lo ambiguo queda como texto.

La capa raw no transforma

Es una copia fiel del origen. No filtra ni descarta filas.

Se paga lo que se lee

BigQuery guarda por columnas y factura los bytes que lee cada query.

El diseño físico sigue a los filtros

Partitioning y clustering se eligen según cómo se va a filtrar la tabla.

Hoy · de la capa raw a las métricas

Cómo se transforman los datos dentro del warehouse y dónde se define cada métrica.

Una pregunta, tres respuestas

¿Cuánto vendimos en septiembre? Tres áreas consultan los mismos datos.

Comercial

$ 48,2 M

Pedidos con fecha de pedido en septiembre

Finanzas

$ 47,4 M

Cobros acreditados en septiembre. Los pedidos de fin de mes se cobran en octubre.

Un analista

$ 47,9 M

Fecha del pedido en UTC. Los pedidos de la noche del 30 caen en octubre.

Las tres son correctas según su definición. Como difieren en 1 o 2 %, nadie lo nota, y nadie sabe cuál usar.

El recorrido de hoy

La clase sigue a los datos desde la capa raw hasta la métrica que consulta el negocio. Cada etapa responde una pregunta de diseño.

1

Modelado y capas

¿Cómo se modelan los datos y qué hace cada capa?

- ETL y ELT
- Modelado dimensional
- Raw, staging y marts
- Tipar, deduplicar y filtrar

2

dbt y métricas

¿Cómo se escriben las transformaciones y dónde se define cada métrica?

- Modelos, source() y ref()
- El linaje y los comandos
- Las materializaciones y el config()
- La capa de métricas

1

Modelado y capas

Cómo se modelan los datos y qué hace cada capa

2

dbt y métricas

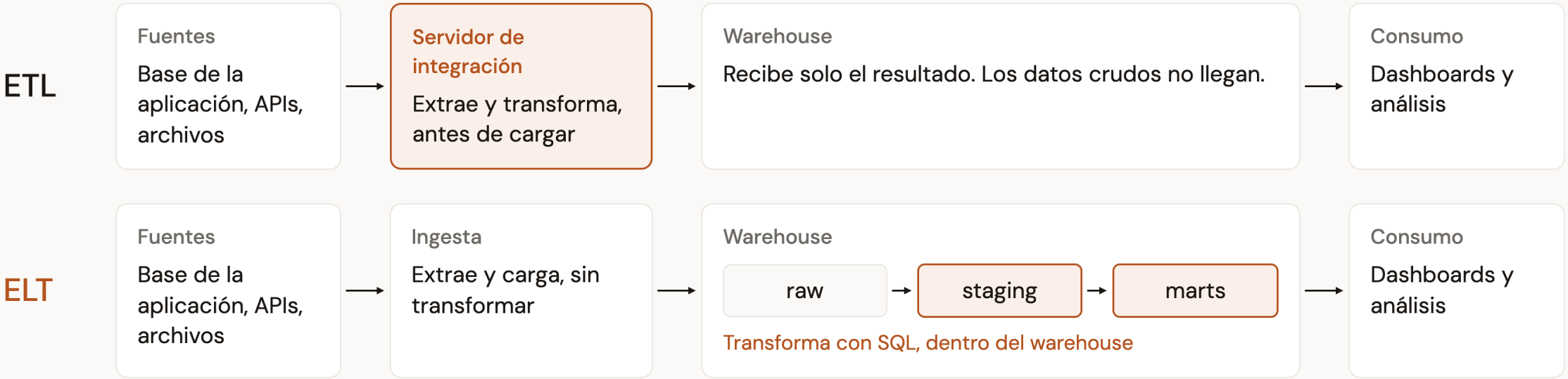
Cómo se escriben las transformaciones y dónde vive cada métrica

Etapa 1 · Modelado y capas

Por qué la transformación ocurre dentro del warehouse, cómo se modelan los datos para analizar, qué responsabilidad tiene cada capa y qué operaciones se hacen en staging y en marts.

La arquitectura de ETL y ELT

Las mismas fuentes y el mismo consumo. Cambian el lugar de la transformación y lo que llega al warehouse.



En naranja, dónde ocurre la transformación.

Las diferencias entre ETL y ELT

Las consecuencias de transformar antes de cargar los datos o después, dentro del warehouse.

	ETL · extraer, transformar, cargar	ELT · extraer, cargar, transformar
Dónde se transforma	En un servidor intermedio, antes de cargar	Dentro del warehouse, con SQL
Qué queda guardado	Solo el resultado	Los datos crudos y el resultado
Si una regla estaba mal	Hay que volver a extraer del origen	Se corrige la regla y se reconstruye desde la capa raw
Quién escribe la lógica	Especialistas en la herramienta de integración	Quien sepa SQL
Herramientas	Informatica, SSIS, DataStage	dbt, Dataform

ETL nació cuando el cómputo del warehouse era caro y escaso. Con almacenamiento y cómputo separados, transformar adentro dejó de ser una carga para el sistema.

Qué es modelar datos

Modelar es decidir cómo se representan los datos para que respondan las preguntas del negocio. Se trabaja en tres niveles, de lo abstracto a lo concreto.

1 · Conceptual

Las entidades del negocio, cómo se relacionan y cuántas de una corresponden a cada una de la otra.

```
Cliente 1 ── N Pedido

Un cliente puede tener muchos
pedidos. Cada pedido es de un
solo cliente.
```

Se discute con el negocio

→

2 · Lógico

Tablas, columnas y la key que identifica cada fila, sin pensar en un motor.

```
pedidos
  pedido_id  key
  cliente_id → clientes
  fecha, monto
```

Lo diseña el equipo de datos

→

3 · Físico

La implementación en un motor concreto: tipos, partición y clustering.

```
fecha DATE
monto NUMERIC
partition by fecha
```

Depende de BigQuery

La normalización

Normalizar es dividir los datos en tablas para que cada dato se guarde en un solo lugar. Se hace aplicando las formas normales.

El problema: un dato en varios lugares

pedidos

pedido_id	cliente_id	nombre	ciudad	monto
1	C01	Ana	Rosario	14.500
2	C02	Luis	Funes	7.700
3	C01	Ana	Rosario	21.100

Una tabla de pedidos que además guarda el nombre y la ciudad del cliente. Los de Ana se repiten en cada uno de sus pedidos.

Al actualizar

Si Ana se muda y queda un pedido sin corregir, Ana tiene dos ciudades.

Al insertar

Un cliente que todavía no compró no tiene dónde guardarse.

Al borrar

Si se borra el pedido 2, se pierde que Luis existe.

Las formas normales más usadas son 1NF, 2NF y 3NF. Cada una agrega una regla sobre de qué depende cada columna.

Normalizado y desnormalizado

Desnormalizar es volver a juntar las tablas y aceptar la redundancia a propósito, a cambio de leer sin joins.

Normalizado · 3NF

clientes

cliente_id	nombre	ciudad
C01	Ana	Rosario
C02	Luis	Funes

pedidos

pedido_id	cliente_id	monto
1	C01	14.500
2	C02	7.700
3	C01	21.100

Cada dato está una sola vez. Para saber la ciudad de un pedido hace falta un join.

Uso típico: OLTP · la base de la aplicación

Desnormalizado

pedidos

pedido_id	monto	cliente_id	nombre	ciudad
1	14.500	C01	Ana	Rosario
2	7.700	C02	Luis	Funes
3	21.100	C01	Ana	Rosario

Se consulta sin joins. El nombre y la ciudad de Ana se repiten en cada uno de sus pedidos.

Uso típico: OLAP · el warehouse

Por qué el warehouse desnormaliza

· Desnormalizar deja los datos juntos en cada slot y evita el shuffle de un join.

· Repetir datos cuesta poco en un almacenamiento por columnas.

· Se escribe una vez y se lee muchas.

· Es más simple de consultar para analistas y BI.

Cuatro enfoques para modelar un warehouse

Son las metodologías de referencia de la industria. En el curso modelamos con Kimball y materializamos una wide table encima para consumir.

Inmon · 3NF centralizado

Primero un warehouse central de toda la empresa, normalizado como una base transaccional. Los marts de cada área salen de ahí.

```
clientes · pedidos · productos
→ mart_ventas · mart_finanzas
```

Ventaja: una sola versión integrada de la empresa
Costo: mucho diseño antes del primer resultado

Kimball · dimensional

Un modelo por proceso de negocio: una tabla de hechos con las medidas y dimensiones que la describen.

```
fct_pedidos
→ dim_clientes · dim_fecha
```

Ventaja: fácil de entender y de consultar
Costo: redundancia en las dimensiones

Data Vault · hubs, links y satellites

Separa las keys, las relaciones y los atributos en tablas distintas. Cada cambio agrega una fila nueva.

```
hub_cliente(cliente_id)
link_pedido_cliente(pedido_id, cliente_id)
sat_cliente(cliente_id, ciudad, desde)
```

Ventaja: guarda toda la historia y absorbe cambios del origen
Costo: complejo y no se consulta directo

Wide table · one big table

Una sola tabla con todo ya unido. Cada fila trae los atributos que necesita.

```
wide_pedidos(pedido_id, fecha, mes,
  cliente_id, ciudad, monto)
```

Ventaja: sin joins y rápida en BigQuery
Costo: mucha redundancia y difícil de mantener sola

El modelo dimensional

Separa lo que se mide de lo que lo describe. Una tabla de hechos en el centro y sus dimensiones alrededor forman un esquema estrella.

dimensión · dim_fecha

fecha	mes	es_fin_de_semana
2026-01-03	1	true
2026-01-05	1	false
2026-01-10	1	true

hechos · fct_pedidos

pedido_id	fecha	cliente_id	monto_neto
1	2026-01-03	C01	14.500
2	2026-01-05	C02	7.700
3	2026-01-10	C01	21.100

dimensión · dim_clientes

cliente_id	es_recurrente
C01	true
C02	false

Hecho

Un evento del negocio por fila, con sus medidas: monto y monto_neto.

Dimensión

El contexto de cada evento, con sus atributos: cuándo fue y quién lo hizo.

Venta de los fines de semana a clientes recurrentes

```
select sum(f.monto_neto) as venta
from fct_pedidos f
join dim_fecha d on d.fecha = f.fecha
join dim_clientes c on c.cliente_id = f.cliente_id
where d.es_fin_de_semana
  and c.es_recurrente
-- 35.600: los pedidos 1 y 3
```

Cómo se diseña un modelo dimensional

Kimball propone cuatro decisiones, en este orden. Aplicadas a los pedidos dan el modelo de la slide anterior.

1

Elegir el proceso de negocio

La actividad que se quiere medir.

La venta

2

Declarar el grano

Qué representa una fila de la tabla de hechos.

Un pedido

3

Identificar las dimensiones

El contexto que describe cada fila.

Fecha y cliente

4

Identificar los hechos

Las medidas numéricas de cada fila.

Monto, descuento y monto neto

El grano se declara antes que las dimensiones y los hechos. Cada dimensión y cada medida tiene que ser coherente con él.

La granularidad de una tabla

La granularidad define qué representa una fila. Es lo primero que se decide de un modelo, y se verifica con una prueba sobre su key.

Modelo	Una fila es	Key
stg_pedidos	Un pedido	pedido_id
fct_pedidos	Un pedido	pedido_id
dim_clientes	Un cliente	cliente_id
dim_fecha	Un día	fecha
wide_pedidos	Un pedido	pedido_id
ventas_diarias	Un día y un medio de pago	fecha y medio_pago

Cuando un join cambia la granularidad

Si la key de la otra tabla se repite, el join multiplica filas.

La query no falla: el total crece sin aviso.

Se previene probando que la key sea única antes del join.

Guardar el grano más fino

Desde un pedido se llega a un día o a un mes. Desde un total diario no se vuelve a los pedidos.

Un solo grano por tabla

El total del día repetido en cada fila de pedido se multiplica al sumarlo.

No toda medida se suma

El promedio de los ticket_promedio diarios no es el ticket promedio del mes.

La wide table

Una tabla con los hechos y los atributos de sus dimensiones ya unidos. Cada fila trae todo lo que una consulta necesita.

dimensión
dim_fecha

hechos
fct_pedidos

dimensión
dim_clientes

→

wide table · wide_pedidos

pedido_id	fecha	es_fin_de_semana	cliente_id	es_recurrente	monto_neto
1	2026-01-03	true	C01	true	14.500
2	2026-01-05	false	C02	false	7.700
3	2026-01-10	true	C01	true	21.100

En naranja, las columnas que vienen de las dimensiones.

models/marts/wide_pedidos.sql

```
select
    f.pedido_id, f.fecha, f.cliente_id, f.monto_neto,
    d.mes, d.es_fin_de_semana,
    c.es_recurrente
from {{ ref('fct_pedidos') }} f
left join {{ ref('dim_fecha') }} d
    on d.fecha = f.fecha
left join {{ ref('dim_clientes') }} c
    on c.cliente_id = f.cliente_id
```

La misma pregunta de la estrella

```
select sum(monto_neto) as venta
from wide_pedidos
where es_fin_de_semana
  and es_recurrente
-- 35.600, sin joins
```

Estrella o wide table

Los dos modelos responden las mismas preguntas. La elección depende de quién consulta y de qué tan seguido cambian los datos.

	Esquema estrella	Wide table
Ventaja principal	Un solo lugar por atributo	Sin joins
Dimensiones compartidas entre varios hechos	•	
Atributos que cambian o con historia	•	
Preguntas nuevas, con SQL	•	
Dashboards con consultas fijas		•
Usuarios de BI sin SQL		•
Variables para machine learning		•

En el curso usamos los dos: la estrella es lo que se mantiene y la wide table es lo que se consulta.

La estrategia de modelado

Cómo se combina lo visto en el proyecto de hoy. Cada paso lee solo del anterior.

1 · Raw

El modelo del origen, tal cual llegó

`pedidos_clase2`

→

2 · Staging

El modelo del origen, con tipos y formatos limpios

`stg_pedidos`

→

3 · Marts

Estrella: hechos y dimensiones

`fct_pedidos`
`dim_clientes`
`dim_fecha`

→

4 · Marts

Wide table y métricas para consumir

`wide_pedidos`
`ventas_diarias`

Raw y staging conservan el modelo del origen. En marts se modela el negocio: la estrella como base y la wide table encima.

10:00

Receso

Las operaciones de cada capa

Lo que hace cada paso de la estrategia, operación por operación. Staging resuelve cada origen por separado; marts combina y agrega.

Raw

`pedidos_clase2`

Ninguna transformación

Se carga tal cual llegó. dbt la declara como source y solo la lee.

→

Staging

`stg_pedidos`

Tipar

La fecha pasa a DATE y el monto a NUMERIC

Normalizar valores

El código postal queda en cuatro dígitos

Tratar los nulos

El descuento vacío pasa a 0

Renombrar columnas

cliente pasa a cliente_id

Deduplicar

Una sola fila por pedido

Filtrar o marcar filas inválidas

Montos negativos, fechas fuera del período

→

Marts

`fct_ · dim_ · wide_pedidos · ventas_diarias`

Separar hechos y dimensiones

fct_pedidos, dim_clientes y dim_fecha

Calcular medidas

El monto neto del descuento

Unir con otras tablas

La wide table con sus dimensiones

Agregar

La venta por día y medio de pago

Convertir tipos sin perder filas

BigQuery ofrece dos formas de convertir un texto a otro tipo. Se diferencian en lo que hacen con un valor que no pueden convertir.

CAST

Un solo valor inválido hace fallar la query entera.

```
cast('1.234,56' as numeric)
-- error: Invalid NUMERIC value
```

SAFE_CAST · SAFE.

El valor inválido se convierte en NULL y la query sigue.

```
safe.parse_date('%d/%m/%Y', '06/01/2026')
-- 2026-01-06
safe.parse_date('%d/%m/%Y', '2026-01-06')
-- NULL
```

SAFE evita que la carga se corte, pero puede generar nulos sin aviso. Por eso cada conversión con SAFE se acompaña de una prueba not_null sobre la columna convertida.

Deduplicar

Antes de sacar duplicados hay que saber por qué existen. Hay dos causas, y cada una se resuelve distinto.

Filas idénticas

La misma carga entró dos veces. Todas las columnas coinciden.

```
select distinct *
from pedidos
```

La misma key, versiones distintas

El origen actualizó el registro. Se conserva la última versión de cada key.

```
select *
from pedidos
qualify row_number() over (
    partition by pedido_id
    order by actualizado_en desc
) = 1
```

Deduplicar construye en staging un grano consistente: una fila por key. Un test unique sobre la key lo confirma.

Filtrar o marcar las filas inválidas

Una fila que no cumple una regla de negocio puede salir del modelo o quedarse con una marca. Es una decisión de diseño.

Filtrar en staging

```
where monto > 0
```

Los marts quedan más simples

Las filas descartadas solo se ven volviendo a raw

Marcar en staging, filtrar en marts

```
monto > 0 as es_valido
```

Se puede contar cuántas filas fallan cada regla

Cada mart tiene que aplicar el filtro

En los dos casos la capa raw conserva todas las filas, y la decisión se puede revisar y reconstruir.

1

Modelado y capas

Cómo se modelan los datos y qué hace cada capa

2

dbt y métricas

Cómo se escriben las transformaciones y dónde vive cada métrica

Etapa 2 · dbt y métricas

Qué es dbt, cómo se arma un proyecto, cómo se encadenan los modelos y qué objeto crea cada uno en BigQuery.
Al final, dónde se define cada métrica.

Qué es dbt

dbt es el framework de transformación del warehouse. Se escriben consultas SELECT, y dbt las convierte en tablas y views de BigQuery, en orden y con tests y documentación.

Qué hace

- Convierte cada SELECT en una tabla o una view, sin escribir DDL
- Ejecuta los modelos en el orden de sus dependencias
- Testea los datos y genera la documentación
- Lleva al SQL las prácticas de software: git, modularidad y CI/CD

Qué no hace

- Extraer ni cargar datos: convive con la ingesta
- Procesar fuera del warehouse: todo corre en BigQuery
- Visualizar: los dashboards leen lo que construye

Versiones

- **dbt v1:** el motor original en Python, open source. Es el que instalaron como dbt-core.
- **dbt v2:** el motor nuevo en Rust, hoy el que se instala por defecto.
- **dbt platform:** la plataforma administrada, antes dbt Cloud, con editor web y ejecuciones programadas.

Cómo está armado un proyecto

Un proyecto de dbt es una carpeta de archivos de texto. Se versiona en git como cualquier otro código.

```
analytics/
├── dbt_project.yml
├── models/
│   ├── staging/
│   │   ├── _sources.yml
│   │   ├── _stg_pedidos.yml
│   │   └── stg_pedidos.sql
│   └── marts/
│       ├── _marts.yml
│       ├── dim_clientes.sql
│       ├── dim_fecha.sql
│       ├── fct_pedidos.sql
│       ├── ventas_diarias.sql
│       └── wide_pedidos.sql
└── tests/
    └── assert_codigo_postal_cuatro_digitos.sql
```

Archivo	Qué define
dbt_project.yml	El nombre del proyecto y la configuración de cada carpeta
models/ · .sql	Un modelo por archivo: un SELECT
models/ · .yml	Sources, descripciones y pruebas genéricas
tests/	Las pruebas singulares
profiles.yml	La conexión a BigQuery. Vive en ~/.dbt, fuera del repositorio

Un modelo es un SELECT

dbt compila el archivo y lo envuelve en el DDL que corresponde a su materialización. El modelo nunca escribe CREATE ni INSERT.

models/staging/stg_pedidos.sql

```sql
select
    pedido_id,
    coalesce(
        safe.parse_date('%Y-%m-%d', fecha),
        safe.parse_date('%d/%m/%Y', fecha)
    ) as fecha,
    cliente as cliente_id,
    cast(
        replace(replace(monto, '.', ''), ',', '.')
        as numeric
    ) as monto,
    medio_pago,
    coalesce(descuento, 0) as descuento_pct,
    regexp_extract(codigo_postal, r'\d{4}')
        as codigo_postal
from {{ source('pedidos', 'pedidos') }}
```

→

Lo que ejecuta en BigQuery

```sql
create or replace view
  `ingenieria-datos-gcp`
  .`dev`.`stg_pedidos`
as
select pedido_id, ...
from `ingenieria-datos-gcp`
  .`pedidos_clase2`
  .`pedidos_demo_schema_declarado`
```

Con la materialización table, el mismo SELECT se envuelve en un CREATE OR REPLACE TABLE.

source() y ref()

Un modelo nunca escribe el nombre completo de una tabla. La pide por su nombre lógico y dbt resuelve dónde está.

source()

Lee una tabla de la capa raw, que dbt no construye. Se declara en un .yml.

```yaml
sources:
  - name: pedidos
    schema: pedidos_clase2
    tables:
      - name: pedidos
        identifier: pedidos_demo_schema_declarado
```

```
{{ source('pedidos', 'pedidos') }}
```

ref()

Lee otro modelo del proyecto. Con cada ref, dbt registra una dependencia.

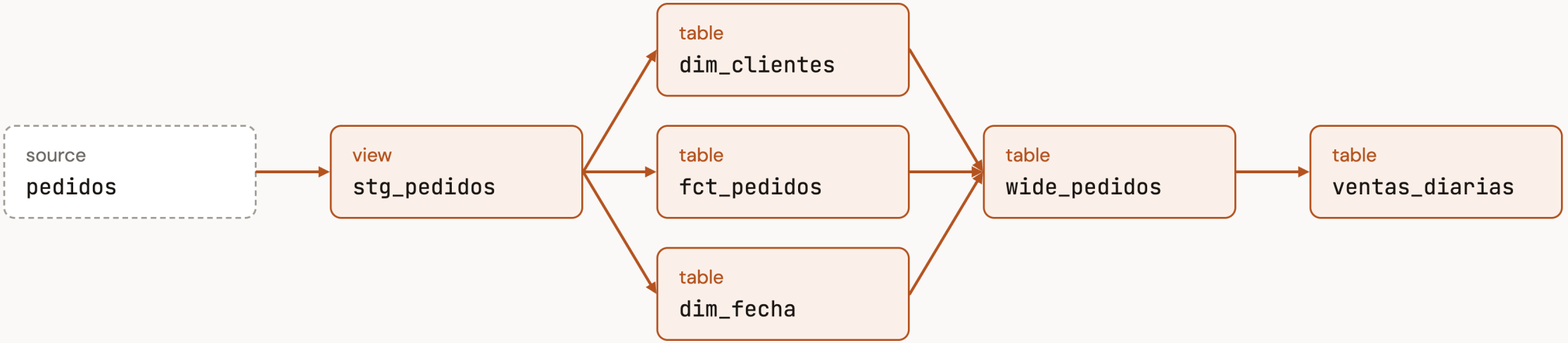
```sql
-- models/marts/fct_pedidos.sql
select
    pedido_id,
    ...
from {{ ref('stg_pedidos') }}
```

Se compila como `` `ingenieria-datos-gcp`.`dev`.`stg_pedidos` ``

Con ref(), el mismo código corre en desarrollo y en producción: cambia solo el dataset del profile.

El linaje

Con cada source() y ref(), dbt arma un grafo dirigido de dependencias. Cada modelo se ejecuta después de los que lee.



Orden de ejecución

dbt run recorre el grafo de izquierda a derecha. Los modelos independientes corren en paralelo.

Impacto de un cambio

Antes de tocar un modelo se sabe qué depende de él, y se puede reconstruir solo esa parte.

Origen de un número

Desde una métrica se llega a la tabla de origen de cada uno de sus datos.

Los comandos de dbt

Se corren desde la carpeta del proyecto, con el entorno virtual activado.

Comando	Qué hace
dbt run	Construye todos los modelos, en el orden del linaje
dbt test	Corre todas las pruebas
dbt build	Construye y prueba cada modelo; si una prueba falla, no construye lo que depende de él
dbt compile	Escribe el SQL compilado en target/, sin ejecutarlo
dbt docs generate	Genera la documentación y el grafo de linaje
dbt run -s stg_pedidos+	Construye stg_pedidos y todos los modelos que dependen de él

Las materializaciones

La materialización define qué objeto crea dbt en BigQuery a partir del SELECT de un modelo.

Materialización	Qué crea	Cuándo se calcula	Cuándo conviene
view	Una view	Cada vez que alguien la consulta	Staging: lógica liviana y pocos lectores
table	Una tabla	Cada vez que corre dbt	Marts: muchas lecturas, joins o agregaciones costosas
incremental	Una tabla que crece	Solo sobre las filas nuevas	Tablas grandes que crecen por período
ephemeral	Ningún objeto	Dentro del modelo que la usa, como CTE	Pasos intermedios que nadie consulta

Una view no ocupa almacenamiento, pero cada consulta paga los bytes de las tablas que lee.

Una tabla paga el almacenamiento, y quien la consulta lee solo el resultado.

La configuración de un modelo

La materialización y el diseño físico se declaran en código: para una carpeta en dbt_project.yml, o para un modelo en su config().

Por carpeta · dbt_project.yml

```yaml
models:
  analytics:
    staging:
      +materialized: view
    marts:
      +materialized: table
```

Por modelo · config() al principio del .sql

```
{{ config(
    materialized='table',
    partition_by={
        'field': 'fecha',
        'data_type': 'date',
        'granularity': 'day'
    },
    cluster_by=['medio_pago']
) }}
```

Si un modelo tiene config(), esa configuración reemplaza a la de su carpeta.

Son las decisiones de la clase 2. Quedan escritas junto al modelo y se versionan con él.

Los modelos incrementales

Un modelo incremental procesa solo las filas nuevas y las agrega a la tabla existente, en lugar de reconstruirla entera en cada corrida.

`table`

Cada corrida lee todo el historial y reescribe la tabla.

`incremental`

La primera corrida construye la tabla. Las siguientes leen solo lo posterior a la última fila cargada y lo agregan con MERGE.

```
{{ config(
    materialized='incremental',
    unique_key='pedido_id'
) }}
select ...
from {{ ref('stg_pedidos') }}
{% if is_incremental() %}
where fecha > (
    select max(fecha) from {{ this }}
)
{% endif %}
```

dbt implementa los modelos incrementales con MERGE, y el sandbox no permite DML. En el curso se ven como concepto y quedan fuera del TP.

La capa de métricas

Cada métrica se define una sola vez, en un modelo versionado y probado. Los dashboards y los análisis la leen de ahí.

models/marts/ventas_diarias.sql

```
select
    fecha,
    medio_pago,
    count(*) as pedidos,
    sum(monto_neto) as venta_neta,
    round(avg(monto_neto), 2)
        as ticket_promedio
from {{ ref('wide_pedidos') }}
group by fecha, medio_pago
```

Métrica	Definición
venta_neta	Suma del monto de cada pedido, neto de su descuento
pedidos	Cantidad de pedidos
ticket_promedio	Promedio del monto neto por pedido

Con la definición escrita en un solo modelo, la venta de septiembre tiene un solo valor para las tres áreas.

Dónde queda escrita cada decisión

Cada decisión de las clases 2 y 3 termina en un archivo del repositorio, en la capa que le corresponde.

Raw

El tipo de cada columna

`pedidos_demo_schema.json`

Staging

La limpieza de cada origen

`stg_pedidos.sql`

El grano de cada tabla

`qualify en stg_pedidos.sql`

Marts

El modelo del negocio

`fct_pedidos · dim_clientes · dim_fecha`

Lo que se consulta

`wide_pedidos.sql`

La definición de cada métrica

`ventas_diarias.sql`

En todas las capas · la materialización y el diseño físico `dbt_project.yml · config()`

Lo que está en el repositorio se revisa, se versiona y se reconstruye desde la capa raw.

La próxima fase del TP

Próxima fase

Staging y marts en dbt

Llevar tp_raw a staging y marts con dbt, con lo visto hoy y en la clase 4.

Mientras tanto

Dejar el entorno listo

Las dos tablas de la parte 1 en tp_raw y dbt debug en verde.

La entrega sigue siendo una sola: miércoles 4 de noviembre, 23:59.

Clase 4 · dbt en la práctica y calidad de datos

27 de octubre, 19:00

Gracias

Preguntas

Google Cloud | The LAD